

Activation-Patching Effects in Controlled CNNs Depend on the Number of Patched Channels, Network Architecture, and First-Layer Filter Structure

Anonymous authors

Paper under double-blind review

Abstract

Activation patching measures how a model output changes when internal activations are replaced, but the result depends on how the intervention is defined. We study this problem in controlled convolutional networks whose first layer is initialized from a predefined bank of edge, corner, and ring filters. Constant regularization keeps the first-layer filters close to the predefined bank, whereas annealing the penalty allows them to move away from it during training. On exact matched counterfactual image pairs, a pre-specified 20-block test on independent renderer blocks shows that the difference in patching effect between release and retention changes sharply with the number of patched first-layer channels; the same pre-specified contrast between $k = 1, 2$ and $k = 4, 8$ remains positive under centered-logit fidelity and unnormalized probability-error reduction, although the pointwise curves differ. A separate 100-block study across 16 CNN architectures finds large differences across architectures under both output metrics, and the same conclusion holds after Greenhouse–Geisser correction, permutation tests, Friedman tests, and random-channel analyses. Finally, a 100-block pixel-permuted control study on independent renderer blocks compares the original filter bank with a common pixel permutation that preserves the template-bank Gram matrix, rank, singular spectrum, row norms, and coefficient multisets. For validation-selected channels, the centered-logit difference averaged across architectures lies outside a pre-specified equivalence margin based on the earlier architecture study, whereas the 90% interval for the probability-error difference lies within its corresponding margin despite excluding zero. The spatial effect reverses sign in one architecture and is substantially smaller on average for random channel sets. Filter similarity alone therefore does not determine the patching effect: the result also depends on the number of patched channels, network architecture, output metric, and the spatial arrangement of the first-layer filters.¹

1 Introduction

The first convolutional layer is one of the few places in a vision network where learned weights can still be inspected directly as spatial filters. This has motivated a long line of work asking whether early spatial filters should be learned freely at all. Wavelet and Gabor constructions provide useful fixed image representations (Bruna & Mallat, 2013). Gabor initialization, maintained filter structure, learnable Gabor parameterizations, and predefined-filter networks impose stronger spatial priors on CNNs (Özbulak & Ekenel, 2018; Molaei & Shiri Ahmad Abadi, 2020; Wang & Alkhalifah, 2024; Linse et al., 2023; Orozco-Solis et al., 2024). Fixed or even random spatial kernels can also remain effective when later layers learn how to recombine them (Gaudio et al., 2023; Gavrikov & Keuper, 2023). Analyses of unconstrained CNNs likewise find regular first-layer frequency structure and sensitivity of early filters to training-data statistics (Chowers & Weiss, 2023; Jorgenson et al., 2026). These results motivate structured first-layer filters, but they leave open a different

¹ChatGPT (OpenAI) was used as an assistive tool during experimental planning, code development and debugging, manuscript organization, and language editing. All experimental protocols, analyses, interpretations, and manuscript text were reviewed by the authors, who take full responsibility for the work.

question: if the first layer remains close to the predefined filters, does that similarity correspond to a stable patching effect?

The distinction matters because representational alignment and effects on model behavior are not interchangeable. Network Dissection, TCAV, Concept Whitening, concept bottlenecks, and related approaches measure or impose different forms of alignment between internal representations and human-defined concepts (Bau et al., 2017; Kim et al., 2018; Chen et al., 2020; Koh et al., 2020). Subsequent work shows that apparently aligned representations can leak unintended information, fail to correspond to intended semantics, or dissociate from effects on model outputs (Margeloiu et al., 2021; Havasi et al., 2022; Marconato et al., 2022; Nicolson et al., 2025; Sharma & Le, 2026). A filter can therefore remain similar to a predefined spatial filter without implying that interventions on the corresponding activation channels behave in a simple or stable way.

Activation patching provides such an intervention: replace an internal activation from one run with the corresponding activation from another and measure the downstream output change. Yet the measured patching effect is not invariant to the experimental setup. Its interpretation depends on patch direction and scope, component granularity, source and target inputs, and output metric (Heimersheim & Nanda, 2024; Zhang & Nanda, 2024). More generally, circuit-faithfulness scores can change under reasonable changes in ablation methodology, so the evaluation procedure partly defines the question being asked (Miller et al., 2024). These concerns are well established in mechanistic-interpretability work, but much less is known about how they interact with explicitly structured first-layer filters in a controlled vision setting.

We use small CNNs and procedural image tasks to isolate this question. Every model begins with the same 16-channel, 9×9 first convolution. One condition is kept near a predefined bank of edge, corner, and ring filters by a constant penalty; another begins at the same bank but is released by annealing that penalty to zero. Exact matched counterfactual image pairs let us replace complete first-layer feature maps using activations from another valid input rather than an artificially corrupted image. We ask three questions. Does the difference in patching effect between release and retention depend on the number of patched channels? With the patching location fixed, does changing the network architecture after the first layer change that difference? Finally, is any such effect specific to the original two-dimensional arrangement of the predefined filters, rather than to regularization toward a fixed filter bank, rank, or other non-spatial properties?

We do not interpret structured filters as inherently interpretable. Instead, we test when preserving their structure changes the measured patching effect. Our main findings are:

1. A pre-specified test on independent renderer blocks shows that the difference in patching effect between release and retention depends strongly on the number of patched first-layer channels. The pre-specified contrast between $k = 1, 2$ and $k = 4, 8$ remains positive under centered-logit fidelity and unnormalized probability error reduction, although the detailed pointwise curves depend on the metric.
2. A separate pre-specified study across 16 CNN architectures shows large differences across architectures under both output metrics. The same conclusion holds after Greenhouse–Geisser correction, within-block permutation tests, Friedman tests, and random channel subsets.
3. A pixel-permuted control experiment on independent renderer blocks isolates first-layer spatial filter structure. Relative to a common pixel permutation that preserves the original bank’s complete Gram matrix, rank, singular spectrum, row norms, and coefficient multisets, the centered-logit difference for validation-selected channels lies outside the pre-specified equivalence margin. That effect is architecture-dependent, reverses sign in one architecture, and is practically much smaller for random channel sets.

Together, these results show that filter similarity does not determine the patching effect, and that channel selection changes the observed effect.

2 Related work

Structured and predefined spatial filters. Hand-designed spatial filters predate modern CNNs, and fixed wavelet scattering shows that substantial invariance and discriminative structure can be obtained without learning arbitrary spatial kernels (Bruna & Mallat, 2013). More recent CNN work has used Gabor banks as initialization (Özbulak & Ekenel, 2018), explicitly maintained Gabor structure during optimization (Molaei & Shiri Ahmad Abadi, 2020), constrained Gabor parameters in task-specific first layers (Wang & Alkhalifah, 2024), and learned texture branches built from Gabor filters (Zhu et al., 2023). Orozco-Solis et al. show that learnable Gabor filters can lose their initial structure over training and use similarity measures to track that degeneration (Orozco-Solis et al., 2024). At a broader architectural level, networks with predefined or fixed spatial filters can retain substantial predictive performance (Linse et al., 2023; Gaudio et al., 2023), while random frozen convolutions can be effective when learned linear combinations are sufficiently expressive (Gavrikov & Keuper, 2023). Analyses of ordinary CNNs further suggest that first-layer energy profiles are highly regular and approximately whitening (Chowers & Weiss, 2023), and that early-filter updates reflect properties of the training distribution (Jorgenson et al., 2026). This literature establishes that first-layer spatial structure can be useful and persistent. Our question is whether preserving one such structure yields a stable patching effect.

Alignment, concepts, and effects on model behavior. Concept-based interpretability methods quantify or impose semantic organization at different representational levels. Network Dissection measures unit-level alignment to visual concepts (Bau et al., 2017); TCAV tests directional sensitivity to user-defined concept vectors (Kim et al., 2018); Concept Whitening rotates latent axes toward specified concepts (Chen et al., 2020); and concept bottleneck models expose an intermediate concept representation that can be edited at test time (Koh et al., 2020). These approaches also expose a recurring limitation: an apparently meaningful representation need not be pure, complete, or used in the intended way. Concept bottlenecks can fail semantic alignment or leak task information (Margeloiu et al., 2021; Havasi et al., 2022; Marconato et al., 2022); concept-vector explanations can vary across layers, become entangled, or depend on spatial position (Nicolson et al., 2025); and recent causal analyses distinguish information that is decodable from information that changes model behavior (Sharma & Le, 2026). We adopt the same distinction at a lower level: filter-template similarity is a structural property, not evidence by itself that interventions on the corresponding activation channels have a stable effect on model behavior.

Activation patching and sensitivity to methodological choices. Direct interventions on visual-network units predate current activation-patching practice: Bau et al. activate or suppress internal CNN/GAN units to test their effect on classification and generated visual content (Bau et al., 2020). Activation patching, interchange interventions, and related causal-abstraction tools generalize the idea of intervening on internal states rather than merely decoding them (Geiger et al., 2025). Practical guidance emphasizes that patching direction, component granularity, input choice, and output metric change what evidence an experiment provides (Heimersheim & Nanda, 2024). Zhang and Nanda show that alternative corruption methods and metrics can yield disparate localization results and caution in particular about non-linear probability metrics and out-of-distribution corruptions (Zhang & Nanda, 2024). Miller et al. similarly show that circuit faithfulness depends strongly on ablation methodology and argue that the task assigned to a circuit cannot be separated from the evaluation procedure (Miller et al., 2024). Broader work questions whether mechanistic explanations are unique or identifiable without additional assumptions (Méloux et al., 2025; Sutter et al., 2025). Our contribution is therefore not the general observation that patching is sensitive to methodological choices. Prior work on structured filters primarily studies filter structure or predictive performance, while prior work on activation patching primarily studies intervention methodology. Here we patch the same first-layer activations, vary how long the predefined filter structure is maintained, measure how the result changes with the number of patched channels and network architecture, and then compare the original two-dimensional arrangement with a control matched on Gram matrix, rank, and singular spectrum.

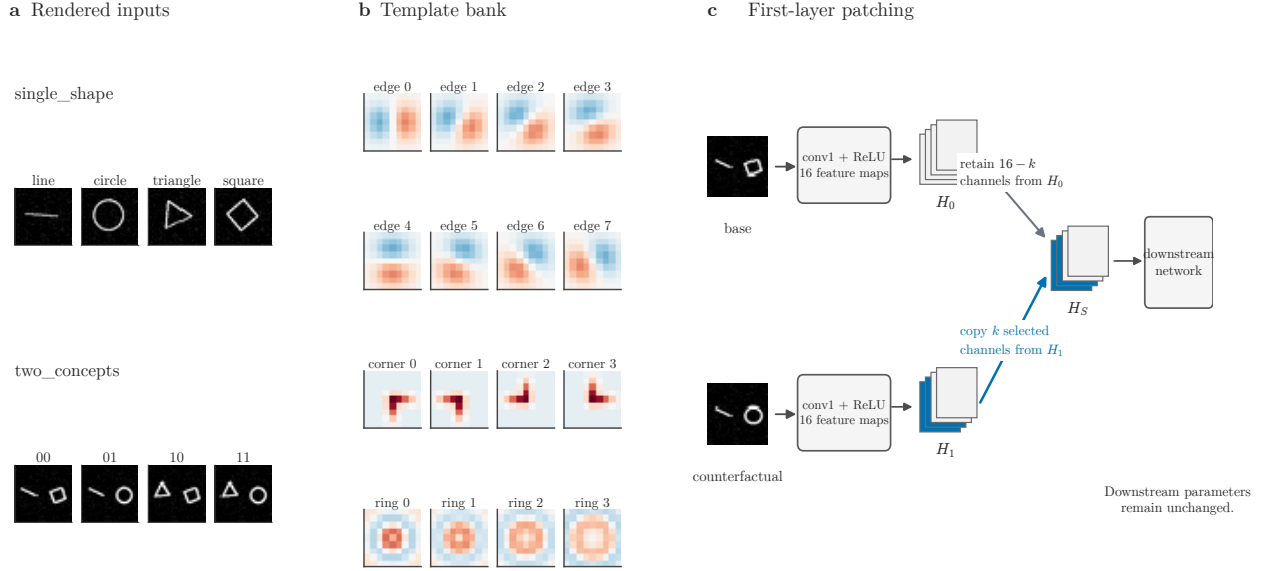


Figure 1: Controlled experimental system. **(a)** Representative inputs from the procedural renderer. **(b)** The 16 predefined first-layer spatial filters: eight edges, four corners, and four rings. **(c)** Activation patching replaces a selected set of k post-ReLU first-layer feature maps in a base run with the corresponding maps from a matched counterfactual run, while downstream parameters remain unchanged. Source activations come from in-distribution counterfactual images; the resulting hybrid activation state is nevertheless an intervention and need not itself lie on the model’s training manifold.

3 Experimental setup

3.1 Controlled tasks and predefined first-layer filters

We use two 32×32 grayscale procedural classification tasks with exact matched counterfactual pairs. Each renderer block contains 512 training, 256 validation, and 256 test images. Rendering varies angle, line width, scale, translation, Gaussian noise, and occasional occlusion. In **single_shape**, the four classes are line, circle, triangle, and square. In **two_concepts**, circle and triangle presence define two binary factors and four classes; a counterfactual pair changes one factor while holding the other factor and nuisance variables fixed. Earlier experiments showed a clearer dependence on the number of patched channels on **two_concepts**; subsequent follow-ups on independent renderer blocks therefore designated **two_concepts** as the primary task and retained **single_shape** as a pre-specified secondary task. This task choice was based on the earlier results but specified before evaluation of each later follow-up.

Every model begins with a bias-free $1 \rightarrow 16$ convolution with 9×9 kernels, same padding, and ReLU. The predefined bank contains eight oriented edges, four two-ray corners, and four rings. Each template is centered and unit norm; the 16×81 flattened bank has numerical rank 10. Figure 1 shows the renderer, the bank, and the intervention point.

For learned filter W_i and template T_j , structural similarity is

$$A = \frac{1}{16} \sum_i \max_j \langle \widehat{W}_i, \widehat{T}_j \rangle, \quad (1)$$

where the inner product is signed cosine similarity after centering and normalizing each filter. A one-to-one matching analysis is retained as a secondary structural check.

3.2 Training conditions and filter-bank controls

Template-based models initialize the first convolution from the predefined filter bank W_{ref} and optimize

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda(e) \frac{\|W - W_{\text{ref}}\|_F^2}{\|W_{\text{ref}}\|_F^2}. \quad (2)$$

The central training comparison uses the same initialization but different regularization schedules. *Retention* keeps $\lambda(e) = 1$ for all 200 epochs. *Release* keeps $\lambda = 1$ through epoch 10, decays linearly to zero by epoch 80, and remains zero thereafter. Adam uses learning rate 0.003 and batch size 128, with no learning-rate scheduler or early stopping. This is a contrast between two complete training regimes, not a post-training intervention that changes only the first-layer kernels while holding the remainder of the trained network fixed: downstream parameters co-adapt to the evolving first layer in each regime.

The pixel-permuted control experiment on independent renderer blocks replaces the original reference filter bank with three alternatives while preserving the same retention/release schedules. The primary control, *pixel-permuted*, applies one common random permutation of the 81 spatial coordinates to every row of the structured bank. Because the same permutation is applied to all rows, it preserves the complete 16×16 Gram matrix, row norms, numerical rank, singular spectrum, and each row’s coefficient multiset while destroying the original two-dimensional edge/corner/ring arrangement. Two secondary controls use a generic random rank-10 bank and a generic random full-rank bank. Random filter-bank realizations are paired across task, architecture, and treatment within each block/initialization replicate.

Network architecture variants. The architecture study keeps the first convolution and patching location fixed and varies only the network after the patched layer. Its 16 variants use global max or global average pooling and controlled convolutional depths of 1, 2, or 4. Additional contrasts vary downstream width (16 or 64 channels for plain networks), residual versus plain depth-4 connectivity, and the presence or absence of downstream BatchNorm. All BatchNorm layers occur after the patched first-layer tensor. The pixel-permuted control study uses a balanced four-architecture subset containing the original depth contrast and the pooling factor that showed a clear difference in the completed architecture study: `tiny_gmp`, `tiny_gap`, `plain2_w16_gmp`, and `plain2_w16_gap`. That four-architecture subset was chosen based on the earlier architecture results but specified before any pixel-permuted control outcome was inspected. Exact definitions and parameter counts are reported in Section A.4.

3.3 Activation patching, ranking, and output metrics

For each matched base/counterfactual pair, we cache the post-ReLU first-layer tensor for both images. A patched run starts from the base image and replaces selected feature maps with the corresponding maps from the counterfactual run. No test-set quantity is used to select channels.

Channels are ranked by the absolute standardized validation-set difference in global-max first-layer activation,

$$d_{g,j} = \frac{\bar{z}_{g=1,j} - \bar{z}_{g=0,j}}{s_j + 10^{-6}}, \quad (3)$$

where z_j is the global maximum of feature map j , s_j is its validation-set standard deviation, and g is the factor changed by the counterfactual. For `two_concepts`, g is the binary circle or triangle factor that is toggled in that counterfactual; for `single_shape`, it is the destination shape identity. Channels are ranked by $|d_{g,j}|$ separately for that concept/identity, and the ranking is fixed using validation data before test patching. The same global-max ranking rule is used for every architecture, including GAP variants, so ranking does not change together with architecture. Eight random channel orders of the same size are evaluated as a control for the validation-based ranking.

Let

$$E_{\text{prob}}(a, b) = \frac{1}{N} \sum_{i=1}^N \|p_a^{(i)} - p_b^{(i)}\|_2^2 \quad (4)$$

denote mean squared probability-space error over N matched pairs. The original patching score is

$$F_{\text{prob}}(S) = 1 - \frac{E_{\text{prob}}(S, 1)}{E_{\text{prob}}(0, 1)}. \quad (5)$$

This score measures normalized reconstruction of the counterfactual output; it is not a mediated fraction or a general measure of causal importance.

Because the denominator in equation (5) varies across models and treatments, later analyses use two alternative output metrics. Centered logits remove the arbitrary common-logit offset,

$$\tilde{\ell} = \ell - \text{mean}(\ell)\mathbf{1}, \quad (6)$$

and centered-logit fidelity applies the same normalized reconstruction form in that space. The unnormalized probability error reduction is

$$R_{\text{prob}}(S) = E_{\text{prob}}(0, 1) - E_{\text{prob}}(S, 1). \quad (7)$$

3.4 Pre-specified contrast and study design

For any patching metric M , define the within-block release-minus-retention difference at channel count k ,

$$\Delta_b^M(k) = M_b^{\text{release}}(k) - M_b^{\text{retention}}(k), \quad (8)$$

and the pre-specified contrast between $k = 1, 2$ and $k = 4, 8$

$$B_b^M = \frac{\Delta_b^M(4) + \Delta_b^M(8)}{2} - \frac{\Delta_b^M(1) + \Delta_b^M(2)}{2}. \quad (9)$$

A positive B means only that the release-minus-retention difference is larger on average at $k = 4, 8$ than at $k = 1, 2$. It does not imply monotonicity in k or a positive release effect at either larger channel count.

Matched image pairs are repeated measurements within a model/block and are never treated as independent replicates. The studies were conducted sequentially rather than specified as one joint preregistration. Earlier analyses motivated a pre-specified 20-block **two_concepts** test of the effect of the number of patched channels in TinyCNN. Its architecture contrast and the pre-specified alternative-metric reanalysis then motivated a separate 100-block architecture study with 16 architecture variants. The architecture results then motivated the pixel-permuted control and the balanced four-architecture subset used on a further 100 independent renderer blocks. Each follow-up protocol and its primary analyses were specified before data from its new renderer blocks were evaluated, although the design of each follow-up was informed by the preceding results.

The 16-architecture and pixel-permuted control studies each average four paired initialization or initialization/filter-bank replicates within renderer block before inference. Their inferential sample size is therefore $n = 100$ renderer blocks, not the 25,600 trained models in each grid. The alternative-metric reanalysis of saved checkpoints and the later Greenhouse–Geisser, permutation, Friedman, and random-channel analyses are post-hoc with respect to the original outcomes, although their analysis definitions were specified before recomputation. Earlier and secondary schedule analyses are retained in the appendix but are not pooled with tests on independent renderer blocks.

4 Results

4.1 Filter similarity does not determine the patching effect

Retention maintains filter similarity as intended. In the larger schedule/task sensitivity sample, **two_concepts**/TinyCNN models under constant regularization have mean template similarity 0.944 versus 0.574 under release, while mean test accuracy is 98.05% versus 99.45%. In the corresponding two-layer model, similarities are 0.999 and 0.896 while accuracies are 99.11% and 99.26%. One-to-one matching gives the same conclusion about filter similarity and rules out the trivial explanation that many learned filters merely match the same few templates (Section E).

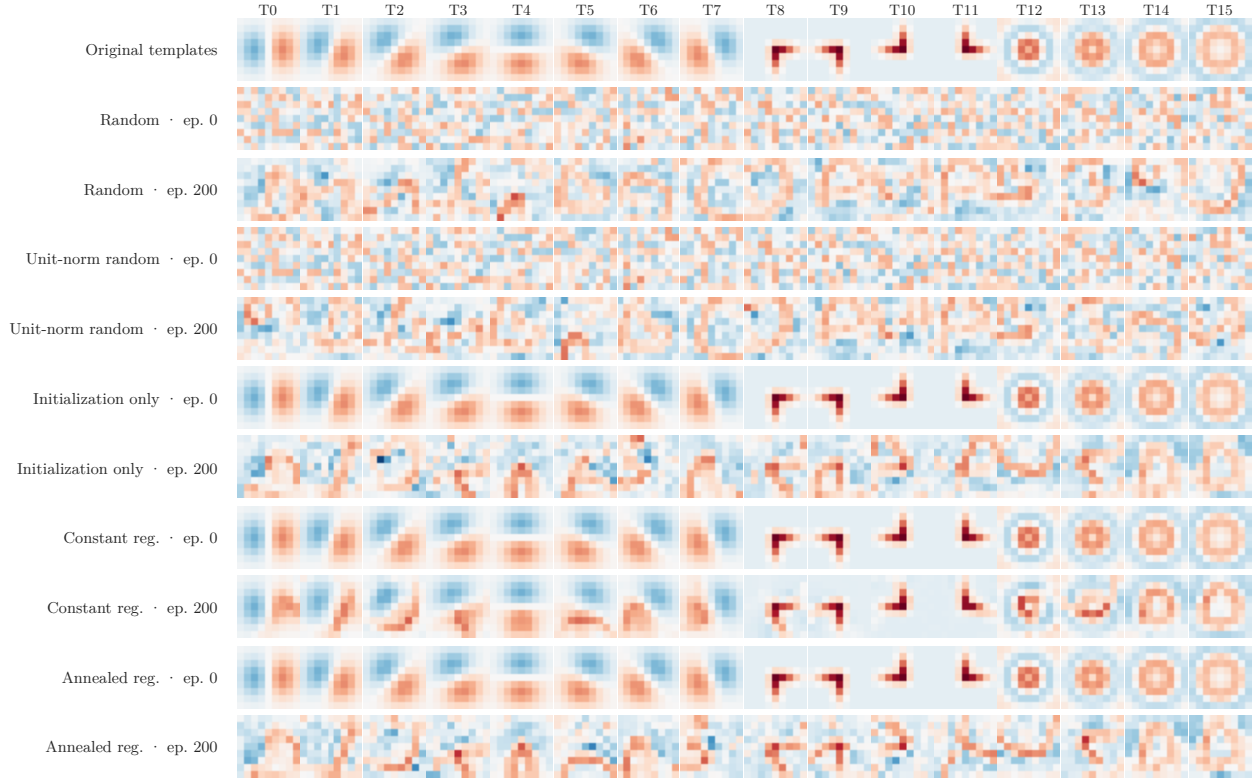


Figure 2: Representative first-layer kernels from a `two_concepts`/TinyCNN checkpoint. The top row shows the predefined bank; later rows show saved filters under the earlier training conditions, reordered by one-to-one template matching for display. Retention preserves visibly template-like kernels, whereas released filters move farther from the bank. The reported statistical results use replicate-level filter-similarity and patching measurements rather than this example.

Figure 2 provides a representative visualization. It is treated as evidence about filter similarity only: neither visual resemblance nor the score in equation (1) establishes semantic purity or an effect on model behavior.

That difference in filter similarity does not determine the activation-patching effect. In the pre-specified 20-block TinyCNN test, the original probability-fidelity contrast is

$$\bar{B}_{\text{prob}} = 0.33098, \quad 95\% \text{ CI} = [0.27368, 0.38827], \quad (10)$$

with $t_{19} = 12.09$, $p = 2.28 \times 10^{-10}$, and all 20 block-level values positive. The pointwise release-minus-retention differences are -0.3459 , -0.3317 , -0.0221 , and $+0.0065$ at $k = 1, 2, 4, 8$, respectively. Thus a single choice of k would describe only one part of the release-minus-retention comparison.

4.2 Channel-count contrast across output metrics

The original probability fidelity has a model-specific denominator, and the pre-specified metric reanalysis shows that this denominator changes substantially between retention and release. The denominator $E_{\text{prob}}(0, 1)$ differs between release and retention by $+0.8486$ on average in TinyCNN, while the corresponding centered-logit denominator differs by $+126.54$. The normalization therefore affects the comparison.

Nevertheless, the pre-specified contrast in equation (9) remains positive under both alternative metrics. Centered-logit fidelity gives $B = +0.17888$ (95% CI $[+0.12727, +0.23049]$, Holm $p = 6.95 \times 10^{-7}$), while probability error reduction gives $B = +0.89441$ (95% CI $[+0.78630, +1.00252]$, Holm $p = 8.59 \times 10^{-13}$).

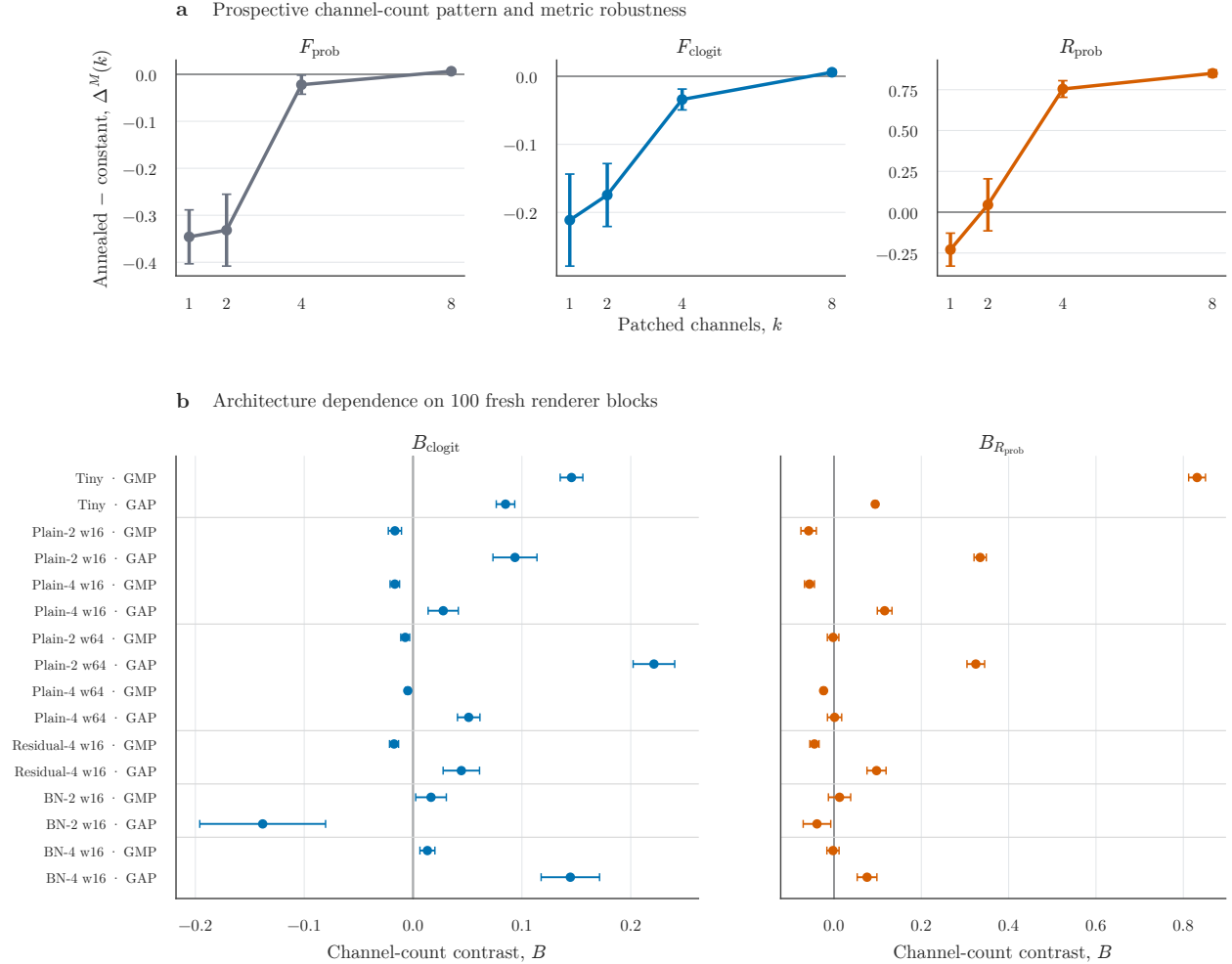


Figure 3: Number of patched channels and network architecture. **(a)** Release-minus-retention differences $\Delta^M(k)$ in the pre-specified `two_concepts`/TinyCNN experiment under the original normalized probability score, centered-logit fidelity, and unnormalized probability error reduction. The three metrics agree on the sign of B but not on the values at individual channel counts. **(b)** Architecture-specific B on the primary `two_concepts` task for 16 pre-specified CNN architectures. Each point averages four initialization replicates within each of 100 independent renderer blocks; bars are marginal 95% t intervals. The two output metrics are shown separately because their numerical scales are not comparable.

Panel (a) of figure 3 shows that the alternative metrics support the same *sign of the pre-specified B contrast*, but not the same values at each k . In particular, unnormalized probability error reduction becomes strongly positive at $k = 4, 8$, whereas the original normalized probability score is close to zero there. The alternative metrics therefore support the same pre-specified contrast, but they are not interchangeable.

4.3 Activation-patching results depend on network architecture

With the first-layer patching procedure, training conditions, validation ranking, counterfactual pairs, and output metrics fixed, changing only the architecture after the first layer produces large differences:

$$F_{15,1485} = 77.09, \quad \eta_p^2 = 0.438 \quad (11)$$

for centered-logit fidelity and

$$F_{15,1485} = 656.70, \quad \eta_p^2 = 0.869 \quad (12)$$

for probability error reduction. On these independent blocks, the TinyGMP-minus-Plain2-GMP contrast is +0.16226 (95% CI [+0.14970, +0.17481]) for centered logits and +0.88960 (95% CI [+0.86212, +0.91707]) for probability error reduction.

The pattern in panel (b) of figure 3 is not a single monotone effect of depth or pooling. Under GMP, most of the Tiny-to-plain shift occurs when the first downstream convolution is added; adding further plain depth changes little. Under GAP, the depth pattern differs. GAP-versus-GMP contrasts are large but reverse direction across backbones. Residual-versus-plain depth-4 contrasts are comparatively small, while BatchNorm effects depend on depth, pooling, and metric. These results show architecture dependence rather than a universal effect of any single architectural component.

Two post-hoc analyses, specified before recomputation, test whether the architecture result depends on sphericity assumptions or on the validation-based channel ranking. First, the architecture effect remains significant after Greenhouse–Geisser correction, 100,000 within-block permutations, and Friedman rank tests; the Holm-adjusted permutation p value is 2.0×10^{-5} for both output metrics. Second, differences across architectures remain when B is computed from the mean over eight random channel orders, again with Holm permutation $p = 2.0 \times 10^{-5}$ for both metrics. The architecture result is therefore not solely produced by the validation-derived channel ranking. Ranking still matters quantitatively: Spearman correlation across architecture means for selected versus random channels is 0.256 for centered logits and 0.724 for probability error reduction (Section A.4).

4.4 Effect of the two-dimensional filter arrangement

We next tested whether the effect depended on the original edge/corner/ring arrangement or arose from regularizing the first layer toward a fixed filter bank more generally. The pixel-permuted control experiment addresses this directly. Its primary control applies the same random permutation of spatial coordinates to every template, preserving the full Gram matrix, numerical rank, singular spectrum, row norms, and row coefficient multisets while destroying the original two-dimensional arrangement.

Averaged over the four architectures on `two_concepts`, selected-channel centered-logit B differs between the structured and pixel-permuted filter banks by +0.05743 (95% CI [+0.05116, +0.06369], Holm sign-flip $p = 2.0 \times 10^{-5}$). Before these outcomes were inspected, the equivalence margin was pre-specified as 20% of the mean absolute B from the earlier architecture study for the same four architectures. The centered-logit 90% interval [+0.05219, +0.06266] lies entirely outside that pre-specified equivalence margin (± 0.01704), so the effect averaged across architectures differs from zero and lies outside the pre-specified equivalence margin.

For probability error reduction, the difference averaged across architectures is +0.04563 (95% CI [+0.03728, +0.05397], Holm sign-flip $p = 2.0 \times 10^{-5}$). Its 90% interval [+0.03864, +0.05261] lies inside the wider pre-specified equivalence margin (± 0.06594). Thus the probability-error contrast averaged across architectures differs from zero, but its 90% interval lies within the pre-specified equivalence margin. This equivalence statement depends on the margin defined from the earlier architecture study; it does not imply a negligible effect of spatial arrangement in every architecture.

The effect also varies strongly across architectures. The structured-minus-pixel-permuted selected-channel effect is positive for `tiny_gmp`, `tiny_gap`, and `plain2_w16_gap`, but negative for `plain2_w16_gmp` under both output metrics. The interaction has partial $\eta^2 = 0.481$ for centered logits and 0.614 for probability error reduction, with Holm permutation $p = 2.0 \times 10^{-5}$ for both. Panel (a) of figure 4 shows the sign reversal directly.

For random channel subsets, the structured-minus-pixel-permuted B averaged across architectures is +0.00571 for centered logits and +0.02007 for probability error reduction, and both effects fall within the pre-specified equivalence margins. These averages are substantially smaller than the corresponding selected-channel centered-logit effect, although architecture-dependent random-channel differences remain detectable. The effect of the original spatial arrangement is therefore larger on average for validation-selected channels than for random subsets, although it is not absent for random subsets.

Finally, the secondary filter-bank controls argue against numerical rank as the main explanation. On the primary task, the pixel-permuted and generic random rank-10 filter banks are generally similar, and differences

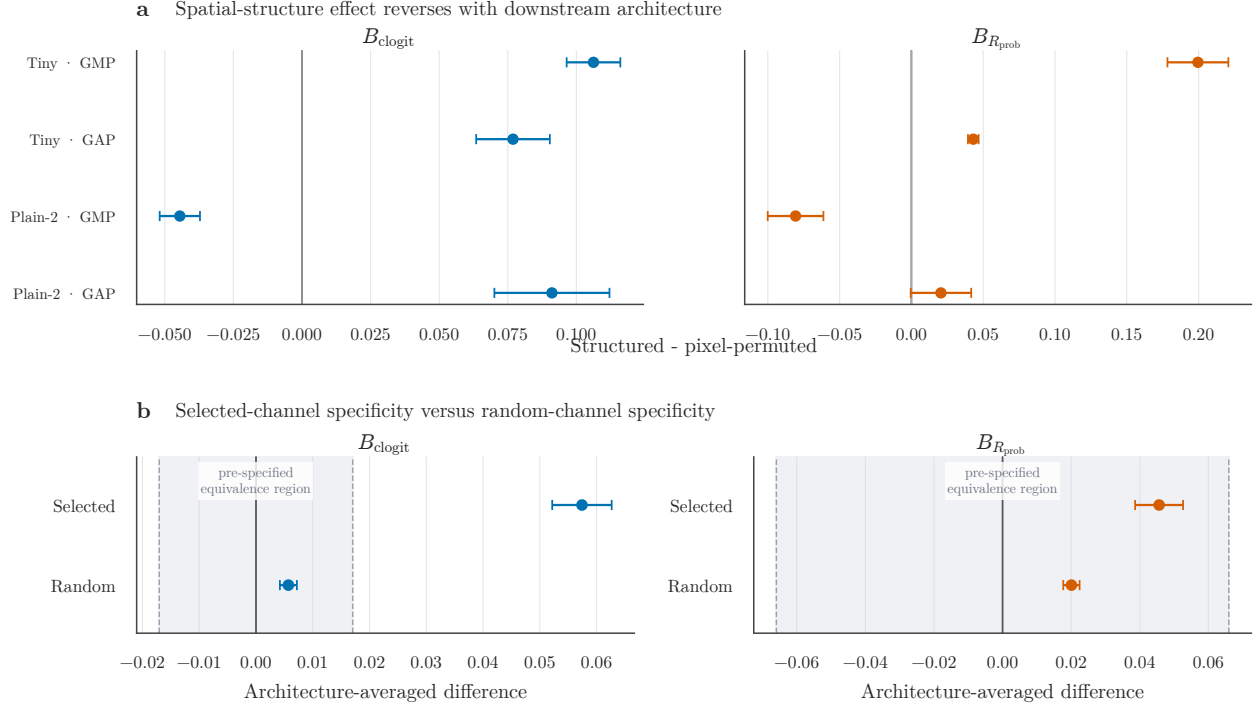


Figure 4: Pixel-permuted control experiment on independent renderer blocks. **(a)** Architecture-specific selected-channel difference in B between the original structured bank and its common pixel-permuted control on `two_concepts`. The spatial effect reverses sign in `plain2_w16_gmp`. Bars are paired 95% t intervals over 100 renderer blocks. **(b)** Structured-minus-pixel-permuted effects averaged across architectures for validation-selected and random channel sets. Bars are 90% intervals used by the pre-specified TOST procedure; shaded regions denote the pre-specified equivalence margins, defined as 20% of the mean absolute B from the earlier architecture study. The selected-channel centered-logit average is not equivalent, whereas the selected-channel probability-error average and both random-channel averages fall within their respective pre-specified margins.

between the random rank-10 and random full-rank filter banks are generally small after the pre-specified family corrections. The original two-dimensional organization is the filter-bank property most clearly isolated by the selected-channel result, but its sign and practical magnitude remain architecture- and metric-dependent.

4.5 Checks and secondary analyses

The pixel-permuted control experiment on independent renderer blocks completes all 25,600 planned training jobs and evaluations. Maximum full-patch and no-op logit identity errors are zero, and the maximum structured-versus-pixel-permuted Gram-matrix discrepancy is 6.66×10^{-16} . The earlier independent checkpoint audit also reproduces central metrics from the earlier analyses at 32 fixed checkpoints within pre-specified numerical tolerances. Secondary schedule, task, ranking, and energy-matched analyses are reported in the appendix as supporting analyses and are not treated as independent replications of the tests on independent renderer blocks.

5 Discussion

Maintaining first-layer filter structure and measuring activation-patching effects answer different questions. Work on predefined, fixed, and Gabor-based filters shows that spatial structure can be useful, robust, or even unnecessary for predictive performance depending on how later layers recombine it (Linse et al., 2023;

Gaudio et al., 2023; Gavrikov & Keuper, 2023; Molaei & Shiri Ahmad Abadi, 2020). Our retention condition confirms that such structure can be maintained almost exactly. But high filter similarity does not determine a unique patching effect. The difference between retention and release patching effects changes with the number of patched channels, and the same first-layer intervention propagates differently through different architectures.

Under centered-logit fidelity, selected-channel B averaged across architectures differs clearly from the pixel-permuted control, which preserves the non-spatial properties of the filter bank. This is not a universal benefit of the original templates: the effect reverses in one architecture, the probability-error contrast averaged across architectures falls within the pre-specified equivalence margin, and the random-channel contrasts averaged across architectures are smaller. These results show that the spatial arrangement changes the release-versus-retention patching effect, with both magnitude and direction depending on architecture, output metric, and channel selection.

The retention-versus-release contrast should also not be read as the isolated causal effect of first-layer geometry in an otherwise fixed trained network. The regularization schedule changes optimization throughout training, and downstream weights co-adapt to the evolving first layer. The comparison therefore concerns two complete training regimes and asks how their activation-patching effects differ.

This distinction parallels a broader lesson from concept-based and mechanistic interpretability. Alignment, decodability, and causal influence answer different questions (Margeloiu et al., 2021; Nicolson et al., 2025; Sharma & Le, 2026). Likewise, activation patching is a family of interventions whose conclusions depend on intervention and evaluation choices (Heimersheim & Nanda, 2024; Zhang & Nanda, 2024; Miller et al., 2024). Our experiments show that these methodological choices also matter in a controlled vision setting with deliberately structured first-layer filters and exact counterfactual pairs. Changing the number of patched channels, network architecture, output metric, first-layer filter structure, or ranking rule changes the quantity being measured.

These results suggest reporting the number and type of patched components, output metric, channel-selection rule, network architecture, and source of replacement activations alongside activation-patching results. Filter similarity should be reported separately from effects measured by activation patching. When a null comparison is scientifically meaningful, equivalence should be tested rather than inferred from nonsignificance. These practices do not make patching a complete mechanistic explanation, but they make the claim being tested substantially clearer.

6 Limitations

- The experiments use two related synthetic image tasks, one family of first-layer spatial templates, and small CNNs. Exact counterfactual pairs provide tight experimental control, but the results do not establish natural-image, large-CNN, vision-transformer, or language-model generalization.
- Patching replaces complete post-ReLU feature maps with activations from an in-distribution matched counterfactual. The source activations are therefore not generated by artificial input corruption, but the hybrid activation state can still be off-manifold for the trained model.
- The validation ranking uses global-max first-layer activation for every architecture, including GAP variants. This keeps ranking fixed while architecture changes, but it may favor some representations. Random-channel analyses show that architecture dependence remains without that ranking, while the pixel-permuted control experiment on independent renderer blocks shows that the effect of spatial arrangement is much smaller for random channels.
- The original normalized probability score has a model-specific denominator that differs across treatments. Later analyses therefore use centered-logit and unnormalized probability metrics. Agreement is strongest at the level of the pre-specified B contrast and architecture omnibus; values at individual channel counts and conclusions about effect size can differ across metrics.
- The probability-error difference between the structured and pixel-permuted banks, averaged across architectures, differs from zero but falls within the pre-specified equivalence margin, defined as 20%

of the mean absolute B from the earlier architecture study. That margin was pre-specified before outcomes on the independent renderer blocks were inspected, but it is a study convention tied to the effect scale in the earlier architecture study rather than an externally validated threshold of scientific importance; architecture-specific effects can be larger and can change sign.

- The studies were conducted sequentially. The architecture study was motivated by earlier outcomes, and the pixel-permuted-control task/architecture subset was chosen after inspecting the architecture results. Each follow-up used a protocol specified before evaluation on independent renderer blocks, but the sequence should not be interpreted as one jointly preregistered experiment.
- Retention versus release compares complete training regimes. Because downstream parameters co-adapt during training, the contrast does not identify the direct causal effect of changing only final first-layer geometry while holding the rest of the trained network fixed.
- Analyses of individual architecture factors and the later Greenhouse–Geisser, Friedman, filter-similarity, and random-channel analyses include tests specified before recomputation, but some are post-hoc with respect to the original architecture outcomes. They are supporting analyses, not independent prospective confirmation.
- Filter-template similarity is a geometric score over filter weights. Neither that score nor the visual appearance of filters establishes human interpretability, semantic purity, a unique mechanism, or an identifiable causal abstraction (Méloux et al., 2025; Sutter et al., 2025).

7 Conclusion

Preserving a predefined first-layer filter bank does not determine the patching effect. In controlled CNNs with exact matched counterfactuals, the release-versus-retention comparison changes with the number of patched channels and varies strongly across architectures after the first layer. The same conclusions hold under alternative output metrics, Greenhouse–Geisser correction, permutation tests, and random-channel analyses. A pixel-permuted control experiment on independent renderer blocks further shows that the original two-dimensional edge/corner/ring arrangement changes the patching effect for validation-selected channels beyond the bank’s Gram matrix, rank, and singular spectrum—but the effect is architecture-dependent, metric-dependent, and smaller on average for random channel sets. Filter similarity and patching effects should therefore be reported separately, together with the channel-selection rule used for the intervention.

8 Data and code availability

The experiments use procedurally rendered synthetic images. The accompanying artifact contains the renderer, model and filter-bank definitions, training and evaluation code, pre-specified protocols, result tables, analysis scripts, checks for the pixel-permuted control, and the independent checkpoint audit. Large raw checkpoints and per-model evaluator outputs are stored separately, while all aggregate data needed to reproduce the reported statistics and figures are versioned. For anonymous review, the manuscript does not link the current working repository because it identifies the authors. An anonymized archive and DOI will be prepared if required by the venue.

References

- David Bau, Bolei Zhou, Aditya Khosla, Aude Oliva, and Antonio Torralba. Network Dissection: Quantifying Interpretability of Deep Visual Representations. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017. URL https://openaccess.thecvf.com/content_cvpr_2017/html/Bau_Network_Dissection_Quantifying_CVPR_2017_paper.html.
- David Bau, Jun-Yan Zhu, Hendrik Strobelt, Agata Lapedriza, Bolei Zhou, and Antonio Torralba. Understanding the Role of Individual Units in a Deep Neural Network. *Proceedings of the National Academy of Sciences*, 2020. doi: 10.1073/pnas.1907375117. URL <https://doi.org/10.1073/pnas.1907375117>.

- Joan Bruna and Stéphane Mallat. Invariant Scattering Convolution Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1872–1886, 2013. doi: 10.1109/TPAMI.2012.230. URL <https://doi.org/10.1109/TPAMI.2012.230>.
- Zhi Chen, Yijie Bei, and Cynthia Rudin. Concept Whitening for Interpretable Image Recognition. *Nature Machine Intelligence*, 2020. doi: 10.1038/s42256-020-00265-z. URL <https://doi.org/10.1038/s42256-020-00265-z>.
- Rhea Chowers and Yair Weiss. What do CNNs Learn in the First Layer and Why? A Linear Systems Perspective. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 6115–6139, 2023. URL <https://proceedings.mlr.press/v202/chowers23a.html>.
- Alex Gaudio, Christos Faloutsos, Asim Smailagic, Pedro Costa, and Aurélio Campilho. ExplainFix: Explainable Spatially Fixed Deep Networks. *WIREs Data Mining and Knowledge Discovery*, 13(2):e1483, 2023. doi: 10.1002/widm.1483. URL <https://doi.org/10.1002/widm.1483>.
- Paul Gavrikov and Janis Keuper. The Power of Linear Combinations: Learning with Random Convolutions. arXiv preprint, 2023. URL <https://arxiv.org/abs/2301.11360v2>.
- Atticus Geiger, Duligur Ibeling, Amir Zur, Maheep Chaudhary, Sonakshi Chauhan, Jing Huang, Aryaman Arora, Zhengxuan Wu, Noah Goodman, Christopher Potts, and Thomas Icard. Causal Abstraction: A Theoretical Foundation for Mechanistic Interpretability. *Journal of Machine Learning Research*, 26(83): 1–64, 2025. URL <https://www.jmlr.org/papers/v26/23-0058.html>.
- Marton Havasi, Sonali Parbhoo, and Finale Doshi-Velez. Addressing Leakage in Concept Bottleneck Models. In *Advances in Neural Information Processing Systems*, 2022.
- Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. arXiv preprint, 2024. URL <https://arxiv.org/abs/2404.15255v1>.
- Grayson Jorgenson, Cassie Heine, Robin Cosbey, Abby Reynolds, Davis Brown, Henry Kvinge, Timothy Doster, and Tegan Emerson. Scratching the Surface: Reflections of Training Data Properties in Early CNN Filters. In *Proceedings of the 1st Conference on Topology, Algebra, and Geometry in Data Science (TAG-DS 2025)*, volume 321 of *Proceedings of Machine Learning Research*, pp. 166–175, 2026. URL <https://proceedings.mlr.press/v321/jorgenson26a.html>.
- Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas, and Rory Sayres. Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 2668–2677, 2018. URL <https://proceedings.mlr.press/v80/kim18d.html>.
- Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept Bottleneck Models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 5338–5348, 2020. URL <https://proceedings.mlr.press/v119/koh20a.html>.
- Christoph Linse, Erhardt Barth, and Thomas Martinetz. Convolutional Neural Networks Do Work with Pre-Defined Filters. In *2023 International Joint Conference on Neural Networks (IJCNN)*, 2023. doi: 10.1109/IJCNN54540.2023.10191449. URL <https://arxiv.org/abs/2411.18388v1>.
- Emanuele Marconato, Andrea Passerini, and Stefano Teso. GlanceNets: Interpretable, Leak-proof Concept-based Models. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2205.15612v2>.
- Andrei Margeloiu, Matthew Ashman, Umang Bhatt, Yanzhi Chen, Mateja Jamnik, and Adrian Weller. Do Concept Bottleneck Models Learn as Intended? Workshop paper at ICLR 2021, 2021. URL <https://arxiv.org/abs/2105.04289v1>.

- Maxime M  loux, Fran  ois Portet, Silviu Maniu, and Maxime Peyrard. Everything, Everywhere, All at Once: Is Mechanistic Interpretability Identifiable? In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2502.20914v1>.
- Joseph Miller, Bilal Chughtai, and William Saunders. Transformer Circuit Faithfulness Metrics Are Not Robust. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=zSf8PJyQb2>.
- Somayeh Molaei and Mohammad Ebrahim Shiri Ahmad Abadi. Maintaining filter structure: A Gabor-based convolutional neural network for image analysis. *Applied Soft Computing*, 88:105960, 2020. doi: 10.1016/j.asoc.2019.105960. URL <https://doi.org/10.1016/j.asoc.2019.105960>.
- Angus Nicolson, Lisa Schut, Alison J. Noble, and Yarin Gal. Explaining Explainability: Recommendations for Effective Use of Concept Activation Vectors. *Transactions on Machine Learning Research*, 2025. URL <https://openreview.net/forum?id=7CUluLpLxV>.
- Carlos Orozco-Solis, Alfonso Rojas-Dom  nguez, H  ctor Puga, Manuel Ornelas Rodr  guez, Mart  n Carpio, and Valent  n Calzada-Ledesma. Learnable Gabor Filters in CNNs: Avoiding Filter Degeneration via Early Stopping Based on Similarity Metrics. In *Pattern Recognition*, volume 14755 of *Lecture Notes in Computer Science*, pp. 387–396, 2024. doi: 10.1007/978-3-031-62836-8_36. URL https://doi.org/10.1007/978-3-031-62836-8_36.
- G  khan   zbulak and Haz  m Kemal Ekenel. Initialization of Convolutional Neural Networks by Gabor Filters. In *2018 26th Signal Processing and Communications Applications Conference (SIU)*, pp. 1–4, 2018. doi: 10.1109/SIU.2018.8404757. URL <https://doi.org/10.1109/SIU.2018.8404757>.
- Aarushi Sharma and Phong Le. Encoding Without Influence: Dissociating Demographic Representation from Causal Effect in Large Language Models. *Transactions on Machine Learning Research*, 2026. URL <https://openreview.net/forum?id=TqbXHsI3Lm>.
- Denis Sutter, Julian Minder, Thomas Hofmann, and Tiago Pimentel. The Non-Linear Representation Dilemma: Is Causal Abstraction Enough for Mechanistic Interpretability? In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2507.08802v2>.
- Fu Wang and Tariq Alkhalifah. Learnable Gabor kernels in convolutional neural networks for seismic interpretation tasks. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–9, 2024. doi: 10.1109/TGRS.2024.3365910. URL <https://doi.org/10.1109/TGRS.2024.3365910>.
- Fred Zhang and Neel Nanda. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2309.16042v2>.
- Lanyun Zhu, Tianrun Chen, Jianxiong Yin, Simon See, and Jun Liu. Learning Gabor Texture Features for Fine-Grained Recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/papers/Zhu_Learning_Gabor_Texture_Features_for_Fine-Grained_Recognition_ICCV_2023_paper.pdf.

Appendix organization. Section A reports the pixel-permuted control experiment and the post-hoc architecture analyses. Section B gives the complete metric- and architecture-robustness summaries. Section C reports the pre-specified channel-count study and the schedule/task sensitivity analyses. Section D preserves the earlier analyses that motivated the pre-specified contrast. Section E reports the complete filter-matching table.

A Pixel-permuted control and architecture analyses

This appendix reports the complete numerical results for the pixel-permuted control experiment and the post-hoc architecture analyses. The control experiment was specified before evaluation on independent renderer blocks; the architecture analyses were performed post-hoc after the original architecture results were known and are labeled accordingly.

A.1 Pixel-permuted control experiment

The experiment uses 100 independent renderer blocks, four initialization/filter-bank replicates per block, two tasks, four architectures, four filter-bank families, and two regularization schedules, for 25,600 trained and evaluated models. The independent unit for inference is the renderer block after averaging the four within-block replicates.

The primary comparison is the original structured edge/corner/ring bank against a bank formed by applying one common random permutation of the 81 spatial coordinates to every template row. This pixel-permuted control preserves, to numerical tolerance, every row norm, the complete 16×16 Gram matrix, numerical rank, singular spectrum, and row coefficient multisets while destroying the original 2D spatial arrangement.

Table 1: Structured-minus-pixel-permuted difference in the selected-channel B contrast, averaged across architectures on `two_concepts`. Difference tests use 100,000 sign-flip permutations and Holm correction across the two output metrics. Equivalence uses the pre-specified TOST margins and 90% confidence intervals.

Metric	Mean difference	95% CI	Holm diff. p	Equivalence result
Centered-logit fidelity	+0.05743	[0.05116, 0.06369]	2.0×10^{-5}	not equivalent ($\delta = 0.01704$)
Probability error reduction	+0.04563	[0.03728, 0.05397]	2.0×10^{-5}	equivalent ($\delta = 0.06594$)

The equivalence margins were pre-specified before the new outcomes as 20% of the mean absolute B from the earlier architecture study for the same four architectures: $\delta_{\text{clogit}} = 0.01704$ and $\delta_{R_{\text{prob}}} = 0.06594$. These are study-specific equivalence margins rather than externally validated thresholds of scientific importance. For centered logits, the 90% interval averaged across architectures is [0.05219, 0.06266] and the Holm-adjusted TOST p value is 1. For probability error reduction, the 90% interval averaged across architectures is [0.03864, 0.05261] and the Holm-adjusted TOST p value is 5.03×10^{-6} . Thus the probability-error difference averaged across architectures is statistically nonzero while falling inside the pre-specified equivalence region. The smallest symmetric margin that would contain its observed 90% interval is approximately 0.05261, about 16.0% of the effect scale from the earlier architecture study used to define the pre-specified 20% margin; this sensitivity is reported to make the role of the margin explicit.

A.2 Architecture dependence of the spatial-arrangement effect

The structured-minus-pixel-permuted effect varies strongly across the four pre-specified architectures.

The corresponding within-block architecture interaction is $F(3, 297) = 91.79$, partial $\eta^2 = 0.481$, Holm permutation $p = 2.0 \times 10^{-5}$ for centered-logit fidelity, and $F(3, 297) = 157.70$, partial $\eta^2 = 0.614$, Holm permutation $p = 2.0 \times 10^{-5}$ for probability error reduction. The sign reversal in `plain2_w16_gmp` rules out a universal positive effect of the original spatial arrangement.

Table 2: Architecture-specific selected-channel structured-minus-pixel-permuted B differences on `two_concepts`. Intervals are paired 95% t intervals over 100 renderer blocks.

Architecture	Centered-logit difference	Probability-error difference
<code>tiny_gmp</code>	+0.10624	+0.19953
<code>tiny_gap</code>	+0.07690	+0.04308
<code>plain2_w16_gmp</code>	-0.04453	-0.08067
<code>plain2_w16_gap</code>	+0.09109	+0.02057

A.3 Random-channel control

Repeating the primary comparison after averaging over eight random channel orders produces much smaller effects of the original spatial arrangement.

Table 3: Structured-minus-pixel-permuted B averaged across architectures for random channel sets on `two_concepts`. Both effects are statistically distinguishable from zero and fall within the pre-specified equivalence margins.

Metric	Mean difference	95% CI
Centered-logit fidelity	+0.00571	[0.00392, 0.00750]
Probability error reduction	+0.02007	[0.01722, 0.02291]

The interaction between filter-bank type and architecture for random channels remains detectable but is smaller for centered logits: partial $\eta^2 = 0.0683$ with permutation $p = 8.0 \times 10^{-5}$. For probability error reduction it remains substantial, partial $\eta^2 = 0.2910$ with permutation $p = 1.0 \times 10^{-5}$. Thus small random-channel effects averaged across architectures coexist with architecture-dependent differences; the equivalence statements apply to the averages across architectures, not to every architecture separately. These are pre-specified secondary analyses.

Secondary filter-bank contrasts show that the pixel-permuted and generic random rank-10 banks are generally similar on the primary task, and that the random rank-10 and random full-rank banks are also generally similar after the pre-specified Holm corrections. The original 2D arrangement, rather than numerical rank alone, is therefore the filter-bank property most clearly distinguished by the selected-channel analysis.

The full-patch and no-op checks give zero logit error, as required. The maximum structured-versus-pixel-permuted Gram-matrix discrepancy is 6.66×10^{-16} .

A.4 Post-hoc architecture analyses

Exact architecture definitions. All variants share the same patched $1 \rightarrow 16$, 9×9 first convolution and ReLU. Pooling is global max (GMP) or global average (GAP). Plain depth-2 networks add one 3×3 convolution; plain depth-4 networks add three. Width-64 variants expand the first downstream convolution to 64 channels and remain width 64 thereafter. Residual depth-4 variants keep width 16 and add the third downstream convolution to the original first-layer activation before the final ReLU. BatchNorm, when present, is downstream of the intervention tensor.

The original 16-architecture study was pre-specified. After seeing its results, separate post-hoc analyses tested whether the omnibus conclusion depended on the uncorrected sphericity assumption or on the validation-based channel ranking.

Differences across architectures also remain when B is computed from the mean over eight random channel orders: Holm-adjusted permutation $p = 2.0 \times 10^{-5}$ for both output metrics. The Spearman correlation across architecture means for selected versus random channels is 0.256 for centered logits and 0.724 for probability error reduction.

Table 4: Exact architecture grid used in the pre-specified architecture study. Parameter counts include the classifier.

Architecture	Conv depth	Width	Pool	BN	Connectivity	Parameters
tiny_gmp	1	16	GMP	no	none	1,364
tiny_gap	1	16	GAP	no	none	1,364
plain2_w16_gmp	2	16	GMP	no	plain	3,684
plain2_w16_gap	2	16	GAP	no	plain	3,684
plain4_w16_gmp	4	16	GMP	no	plain	8,324
plain4_w16_gap	4	16	GAP	no	plain	8,324
plain2_w64_gmp	2	64	GMP	no	plain	10,836
plain2_w64_gap	2	64	GAP	no	plain	10,836
plain4_w64_gmp	4	64	GMP	no	plain	84,692
plain4_w64_gap	4	64	GAP	no	plain	84,692
res4_w16_gmp	4	16	GMP	no	residual	8,324
res4_w16_gap	4	16	GAP	no	residual	8,324
bn2_w16_gmp	2	16	GMP	yes	plain	3,700
bn2_w16_gap	2	16	GAP	yes	plain	3,700
bn4_w16_gmp	4	16	GMP	yes	plain	8,372
bn4_w16_gap	4	16	GAP	yes	plain	8,372

Table 5: Sphericity-corrected and permutation-based architecture omnibus analyses on the primary **two_concepts** task. Permutation p values use 100,000 within-block permutations and are Holm adjusted across the two metrics within each analysis kind.

Metric	GG ϵ	GG p	Permutation	Holm p	Friedman p
Centered-logit fidelity	0.1861	1.56×10^{-34}		2.0×10^{-5}	3.52×10^{-146}
Probability error reduction	0.5973	$< 10^{-300}$		2.0×10^{-5}	7.79×10^{-200}

For the TinyGMP-minus-Plain2-GMP contrast using random channel sets, the mean difference is +0.002765 (95% CI [+0.000014, +0.005515], Holm p = 0.0488) for centered logits and +0.319871 (95% CI [+0.312788, +0.326953], Holm p = 2.76×10^{-96}) for probability error reduction. The architecture conclusion is therefore not solely a consequence of the selected-channel ranking, although the ranking changes effect magnitude and the ordering across architectures.

B Metric sensitivity and architecture results

This appendix gives the complete numerical results for the pre-specified metric-sensitivity reanalysis and the pre-specified architecture study. Block-level values, supporting tables, and control analyses are versioned under `analysis/metric_sensitivity_001/` and `analysis/architecture_robustness_001/`. The main text reports the principal estimates.

B.1 Metric-sensitivity analysis

The metric-sensitivity analysis reuses saved checkpoints from the pre-specified channel-count and schedule/task sensitivity experiments. It was designed after the original probability-fidelity results were known, so it is post-hoc with respect to those results; however, the alternative metric definitions and analysis plan were specified before their values were computed. No model was retrained and the original validation-derived channel rankings were reused.

For the primary `two_concepts`/TinyCNN setting, Table 6 gives the full pointwise annealed-minus-constant curves under the two alternative output metrics.

Table 6: TinyCNN pointwise release-minus-retention contrasts under the pre-specified alternative metrics.

Metric	k	Mean Δ	95% CI
Centered-logit fidelity	1	-0.21134	[-0.27899, -0.14369]
Centered-logit fidelity	2	-0.17452	[-0.22078, -0.12825]
Centered-logit fidelity	4	-0.03405	[-0.04939, -0.01871]
Centered-logit fidelity	8	+0.00595	[+0.00253, +0.00937]
Probability error reduction	1	-0.22942	[-0.33033, -0.12852]
Probability error reduction	2	+0.04493	[-0.11443, +0.20428]
Probability error reduction	4	+0.75429	[+0.70327, +0.80530]
Probability error reduction	8	+0.85004	[+0.82915, +0.87093]

The corresponding B contrasts are +0.178881 (95% CI [+0.127268, +0.230494]) for centered-logit fidelity and +0.894411 (95% CI [+0.786301, +1.002522]) for probability error reduction. Holm-adjusted p values for the two-metric family are 6.95×10^{-7} and 8.59×10^{-13} , respectively.

The release-minus-retention base-to-counterfactual error scales are themselves different. In TinyCNN the probability input-effect-scale contrast is +0.848558 (95% CI [+0.826701, +0.870416]), and the centered-logit input-effect-scale contrast is +126.537 (95% CI [+120.845, +132.229]). In TwoLayerCNN the corresponding contrasts are +0.062062 and +211.368. Thus the original normalized metric cannot be treated as though its denominator were constant across treatments.

The same pre-specified analysis applied to the schedule/task sensitivity experiment gives the default-release results in Table 7.

Table 7: Schedule/task sensitivity default-release B contrasts under the pre-specified alternative metrics. Values are release minus constant $\lambda = 1$ under the original activation-contrast ranking.

Task	Architecture	B_{clogit}	$B_{R_{\text{prob}}}$
single shape	TinyCNN	+0.05660	+0.33747
single shape	TwoLayerCNN	+0.05700	-0.01622
two concepts	TinyCNN	+0.13427	+0.80239
two concepts	TwoLayerCNN	-0.01953	-0.05497

The maximum absolute discrepancy between recomputed and stored original probability fidelity is 1.24×10^{-7} . Maximum full-patch and no-op logit identity errors are zero.

B.2 Architecture results on the primary task

The architecture study uses 100 independent renderer blocks, four paired initialization replicates per block, 16 CNN architectures, two tasks, and the two central training conditions. Initialization replicates are averaged before inference. The primary task is **two_concepts**; the independent unit for inference is therefore the renderer block ($n = 100$) for each architecture-level comparison.

Table 8: Repeated-measures architecture omnibus tests on **two_concepts**.

Metric	F	df	partial η^2	Holm p
centered_logit_fidelity	77.09	15, 1485	0.438	2.40×10^{-173}
prob_error_reduction	656.70	15, 1485	0.869	0 (machine precision)

For probability error reduction, the computed upper-tail p value is numerically zero at double precision; this is reported as a machine-precision underflow rather than a literal probability of zero.

Table 9: TinyGMP-minus-Plain2-GMP architecture contrast, **tiny_gmp** minus **plain2_w16_gmp**.

Metric	Mean difference in B	95% CI	Holm p
centered_logit_fidelity	+0.16226	[+0.14970, +0.17481]	1.66×10^{-45}
prob_error_reduction	+0.88960	[+0.86212, +0.91707]	3.24×10^{-82}

B.3 Architecture-specific estimates

Table 10 reports every architecture mean and marginal 95% t interval for the two pre-specified output metrics.

Table 10: **two_concepts** architecture means for B .

Architecture	B_{clogit}	95% CI	$B_{R_{\text{prob}}}$	95% CI
tiny_gmp	+0.1456	[+0.1352, +0.1560]	+0.8315	[+0.8122, +0.8508]
tiny_gap	+0.0849	[+0.0764, +0.0934]	+0.0944	[+0.0901, +0.0986]
plain2_w16_gmp	-0.0167	[-0.0227, -0.0106]	-0.0581	[-0.0757, -0.0404]
plain2_w16_gap	+0.0936	[+0.0734, +0.1139]	+0.3348	[+0.3207, +0.3488]
plain4_w16_gmp	-0.0168	[-0.0211, -0.0124]	-0.0561	[-0.0675, -0.0447]
plain4_w16_gap	+0.0277	[+0.0138, +0.0416]	+0.1160	[+0.0992, +0.1328]
plain2_w64_gmp	-0.0072	[-0.0113, -0.0031]	-0.0021	[-0.0152, +0.0111]
plain2_w64_gap	+0.2213	[+0.2022, +0.2404]	+0.3247	[+0.3045, +0.3449]
plain4_w64_gmp	-0.0049	[-0.0068, -0.0029]	-0.0238	[-0.0282, -0.0194]
plain4_w64_gap	+0.0511	[+0.0408, +0.0614]	+0.0013	[-0.0150, +0.0176]
res4_w16_gmp	-0.0175	[-0.0216, -0.0133]	-0.0448	[-0.0554, -0.0342]
res4_w16_gap	+0.0444	[+0.0276, +0.0611]	+0.0974	[+0.0756, +0.1192]
bn2_w16_gmp	+0.0165	[+0.0024, +0.0306]	+0.0127	[-0.0128, +0.0382]
bn2_w16_gap	-0.1382	[-0.1960, -0.0804]	-0.0390	[-0.0706, -0.0074]
bn4_w16_gmp	+0.0132	[+0.0063, +0.0201]	-0.0023	[-0.0163, +0.0116]
bn4_w16_gap	+0.1445	[+0.1177, +0.1713]	+0.0757	[+0.0532, +0.0983]

B.4 Pre-specified architecture-factor contrasts

Table 11 gives all pre-specified factor contrasts on the primary **two_concepts** task. Holm adjustment is applied within metric, task, and factor family exactly as specified in the architecture protocol. The signs are alternative architecture minus reference architecture as named in each row.

Table 11: Architecture-factor contrasts.

Family	Contrast	ΔB_{clogit}	Holm p	$\Delta B_{R_{\text{prob}}}$	Holm p
pooling	tiny: GAP - GMP	-0.0607	7.25×10^{-14}	-0.7372	4.41×10^{-88}
pooling	plain2_w16: GAP - GMP	+0.1103	3.42×10^{-17}	+0.3929	3.70×10^{-56}
pooling	plain4_w16: GAP - GMP	+0.0445	7.53×10^{-8}	+0.1721	4.57×10^{-28}
pooling	plain2_w64: GAP - GMP	+0.2285	5.03×10^{-40}	+0.3268	2.56×10^{-46}
pooling	plain4_w64: GAP - GMP	+0.0560	1.60×10^{-17}	+0.0251	0.0073
pooling	res4_w16: GAP - GMP	+0.0618	2.17×10^{-10}	+0.1422	3.48×10^{-19}
pooling	bn2_w16: GAP - GMP	-0.1547	1.17×10^{-6}	-0.0517	0.0146
pooling	bn4_w16: GAP - GMP	+0.1313	4.09×10^{-15}	+0.0780	1.34×10^{-7}
depth	GMP: plain2 - tiny	-0.1623	6.62×10^{-45}	-0.8896	6.48×10^{-82}
depth	GAP: plain2 - tiny	+0.0087	0.6905	+0.2404	4.30×10^{-57}
depth	GMP: plain4 - plain2	-0.0001	0.9845	+0.0019	0.8552
depth	GAP: plain4 - plain2	-0.0659	6.12×10^{-7}	-0.2188	1.41×10^{-36}
width	depth2 GMP: w64 - w16	+0.0095	0.0095	+0.0560	1.40×10^{-6}
width	depth2 GAP: w64 - w16	+0.1277	1.51×10^{-17}	-0.0100	0.3637
width	depth4 GMP: w64 - w16	+0.0119	2.14×10^{-6}	+0.0323	1.82×10^{-6}
width	depth4 GAP: w64 - w16	+0.0234	0.0095	-0.1147	8.27×10^{-16}
residual	GMP: residual4 - plain4	-0.0007	0.7775	+0.0113	0.1961
residual	GAP: residual4 - plain4	+0.0166	0.1943	-0.0186	0.1961
batchnorm	depth2 GMP: BN - noBN	+0.0332	1.09×10^{-5}	+0.0708	3.31×10^{-5}
batchnorm	depth2 GAP: BN - noBN	-0.2318	2.67×10^{-11}	-0.3738	1.92×10^{-37}
batchnorm	depth4 GMP: BN - noBN	+0.0299	4.90×10^{-11}	+0.0538	2.61×10^{-7}
batchnorm	depth4 GAP: BN - noBN	+0.1168	2.67×10^{-11}	-0.0403	0.0036

The residual-versus-plain contrasts give the clearest null results: all four metric/pooling combinations have intervals spanning zero after the pre-specified Holm correction. Pooling, depth, width, and BatchNorm show multiple nonzero contrasts, but directions vary by backbone and metric. The data therefore support architecture dependence without supporting a single architecture-independent component effect.

B.5 Secondary `single_shape` architecture results

The secondary task also shows substantial variation across architectures, but its sign pattern differs in several settings. Table 12 is included to make that task dependence visible rather than pooling the two tasks.

Table 12: `single_shape` mean B by architecture.

Architecture	B_{clogit}	$B_{R_{\text{prob}}}$
tiny_gmp	+0.0569	+0.3459
tiny_gap	-0.0135	+0.2644
plain2_w16_gmp	+0.0525	-0.0203
plain2_w16_gap	+0.0449	+0.2026
plain4_w16_gmp	+0.0143	+0.0115
plain4_w16_gap	+0.1551	+0.1475
plain2_w64_gmp	+0.0321	-0.0463
plain2_w64_gap	+0.0432	+0.1525
plain4_w64_gmp	+0.0021	+0.0015
plain4_w64_gap	+0.0808	+0.0923
res4_w16_gmp	+0.0139	+0.0194
res4_w16_gap	+0.1539	+0.1241
bn2_w16_gmp	+0.0668	+0.1155
bn2_w16_gap	-0.2701	-0.0340
bn4_w16_gmp	+0.0028	+0.0102
bn4_w16_gap	+0.3773	+0.0383

For the architecture experiment, the maximum full-patch and no-op logit identity errors are zero. Initialization-dispersion summaries, denominator contrasts, random-channel controls, replacement-energy controls, and block-level values are retained in the versioned analysis tables.

C Pre-specified channel-count and schedule/task sensitivity analyses

This appendix reports the full numerical results for the pre-specified channel-count experiment and the subsequent schedule/task sensitivity experiment. The channel-count protocol was specified after the earlier analyses but before running blocks 4000–4019, which are disjoint from the earlier data. It was not externally preregistered. The schedule/task sensitivity protocol was separately specified before running blocks 5000–5049 and remains secondary to the channel-count test. Blocks are independent experimental replicates; conditions within a block are paired through shared generated data and relevant initialization components. The main text reports the primary channel-count result and uses the schedule/task analysis as a secondary sensitivity analysis. This appendix preserves the complete channel-selection and schedule breakdowns, secondary controls, and reimplementations details. The complete tables are retained under `analysis/budget_confirmation_001/` and `analysis/exhaustive_robustness_001/`.

C.1 Pre-specified selected-channel patching results

Each entry is the within-block difference in patching effect for selected channels at epoch 200 (annealed minus constant regularization), averaged over 20 independent blocks. Only TinyCNN with channels ranked by activation difference supplies the single primary pre-specified test; the remaining rows are pre-specified secondary analyses.

Table 13: Annealed-minus-constant differences in patching effect for selected channels.

Architecture	Channel selection	k	Mean ΔF	95% t interval
TinyCNN	activation difference	1	-0.3459	[-0.4033, -0.2885]
TinyCNN	activation difference	2	-0.3317	[-0.4082, -0.2552]
TinyCNN	activation difference	4	-0.0221	[-0.0424, -0.0019]
TinyCNN	activation difference	8	+0.0065	[+0.0037, +0.0093]
TinyCNN	AUROC	1	-0.3040	[-0.3662, -0.2418]
TinyCNN	AUROC	2	-0.3237	[-0.3988, -0.2485]
TinyCNN	AUROC	4	-0.0332	[-0.0696, +0.0032]
TinyCNN	AUROC	8	+0.0058	[+0.0026, +0.0091]
TinyCNN	validation patching	1	-0.2856	[-0.3191, -0.2520]
TinyCNN	validation patching	2	-0.1733	[-0.2068, -0.1399]
TinyCNN	validation patching	4	-0.0052	[-0.0202, +0.0098]
TinyCNN	validation patching	8	+0.0073	[+0.0047, +0.0098]
TwoLayerCNN	activation difference	1	-0.0117	[-0.0202, -0.0032]
TwoLayerCNN	activation difference	2	-0.0378	[-0.0493, -0.0263]
TwoLayerCNN	activation difference	4	-0.1313	[-0.1645, -0.0980]
TwoLayerCNN	activation difference	8	-0.0020	[-0.0493, +0.0453]
TwoLayerCNN	AUROC	1	-0.0141	[-0.0224, -0.0058]
TwoLayerCNN	AUROC	2	-0.0438	[-0.0565, -0.0311]
TwoLayerCNN	AUROC	4	-0.1319	[-0.1614, -0.1025]
TwoLayerCNN	AUROC	8	+0.0104	[-0.0402, +0.0611]
TwoLayerCNN	validation patching	1	-0.0256	[-0.0380, -0.0131]
TwoLayerCNN	validation patching	2	-0.0760	[-0.1048, -0.0473]
TwoLayerCNN	validation patching	4	-0.0883	[-0.1401, -0.0365]
TwoLayerCNN	validation patching	8	+0.0100	[-0.0015, +0.0214]

C.2 Pre-specified channel-count contrast

For every architecture/channel-selection pair,

$$B = \frac{\Delta F(4) + \Delta F(8)}{2} - \frac{\Delta F(1) + \Delta F(2)}{2}. \quad (13)$$

Positive B indicates that the annealed-minus-constant difference is larger on average at $k = 4, 8$ than at $k = 1, 2$; it does not imply monotonicity in k and does not require a positive annealing effect at either larger value. The first TinyCNN row, with channels ranked by activation difference, is the single pre-specified primary test. Its 20 block-level B values are all positive (range 0.1452–0.5986). All other rows are secondary and do not alter the primary conclusion.

Table 14: Values of the pre-specified contrast B . The first row alone is the primary pre-specified test.

Architecture	Channel selection	Mean B	95% CI	two-sided p
TinyCNN	activation difference	+0.33098	[0.27368, 0.38827]	2.28×10^{-10}
TinyCNN	AUROC	+0.30014	[0.24866, 0.35162]	1.95×10^{-10}
TinyCNN	validation patching	+0.23050	[0.20442, 0.25658]	1.31×10^{-13}
TwoLayerCNN	activation difference	-0.04189	[-0.07010, -0.01367]	0.00580
TwoLayerCNN	AUROC	-0.03179	[-0.05830, -0.00529]	0.02126
TwoLayerCNN	validation patching	+0.01162	[-0.00949, 0.03273]	0.26368

C.3 Patched-run accuracy on counterfactual targets

Patched-run accuracy on the counterfactual target was retained alongside the normalized patching effect. For the primary TinyCNN setting, with channels ranked by activation difference, annealed minus constant regularization is -0.2009 at $k = 1$, -0.3626 at $k = 2$, -0.0241 at $k = 4$, and $+0.0140$ at $k = 8$. The corresponding 95% intervals are $[-0.2551, -0.1466]$, $[-0.4400, -0.2852]$, $[-0.0524, 0.0042]$, and $[0.0086, 0.0193]$.

AUROC and validation-patching channel-selection rules show the same broad change between smaller and larger channel sets. These are secondary outcomes and were not used to define the primary success criterion.

C.4 Energy-matched control

The energy-matched control uses validation replacement energy to select eight alternative channel sets of the same size whose replacement energy is closest to that of the selected set, using validation data only. For TinyCNN with channels ranked by activation difference, the annealed-minus-constant difference in U_{energy} is -0.2980 at $k = 1$, -0.1887 at $k = 2$, $+0.1310$ at $k = 4$, and $+0.0696$ at $k = 8$. Matching quality is not uniform: single-channel alternatives can be poorly matched, whereas $k = 4$ and $k = 8$ are generally much tighter. This analysis shows that the difference relative to the energy-matched baseline also changes with the number of patched channels, but the primary analysis remains the patching effect for selected channels.

C.5 Schedule sensitivity when channels are ranked by activation difference

The main text uses the schedule/task experiment only as a secondary sensitivity analysis. The table below gives all five schedule-versus-constant-regularization comparisons for each task/architecture setting. Holm adjustment is performed within each task/architecture/channel-selection family, exactly as pre-specified.

Table 15: Schedule-sensitivity values of B for every non-reference regularization schedule when channels are ranked by activation difference.

Task	Architecture	Schedule vs. constant $\lambda = 1$	Mean B	95% CI	Holm p
single shape	TinyCNN	initialization only	+0.06804	[0.05343, 0.08264]	6.91×10^{-12}
single shape	TinyCNN	constant $\lambda = 0.1$	+0.03871	[0.02525, 0.05218]	5.13×10^{-7}
single shape	TinyCNN	early annealing	+0.06855	[0.05228, 0.08482]	1.11×10^{-10}
single shape	TinyCNN	default annealing	+0.07189	[0.05444, 0.08934]	1.43×10^{-10}
single shape	TinyCNN	late annealing	+0.07955	[0.06404, 0.09505]	3.61×10^{-13}
single shape	TwoLayerCNN	initialization only	+0.00762	[-0.00793, 0.02317]	0.9885
single shape	TwoLayerCNN	constant $\lambda = 0.1$	+0.00031	[-0.01053, 0.01116]	1.0000
single shape	TwoLayerCNN	early annealing	+0.00003	[-0.01500, 0.01506]	1.0000

Task	Architecture	Schedule vs. constant $\lambda = 1$	Mean B	95% CI	Holm p
single shape	TwoLayerCNN	default annealing	-0.01018	[-0.02290, 0.00254]	0.4569
single shape	TwoLayerCNN	late annealing	-0.01237	[-0.02194, -0.00279]	0.0621
two concepts	TinyCNN	initialization only	+0.34557	[0.31797, 0.37317]	5.61×10^{-29}
two concepts	TinyCNN	constant $\lambda = 0.1$	+0.16626	[0.14002, 0.19251]	3.71×10^{-17}
two concepts	TinyCNN	early annealing	+0.32245	[0.28415, 0.36074]	1.26×10^{-21}
two concepts	TinyCNN	default annealing	+0.28619	[0.25389, 0.31850]	1.95×10^{-22}
two concepts	TinyCNN	late annealing	+0.19342	[0.16432, 0.22253]	1.20×10^{-17}
two concepts	TwoLayerCNN	initialization only	-0.00583	[-0.03214, 0.02049]	0.6583
two concepts	TwoLayerCNN	constant $\lambda = 0.1$	-0.01413	[-0.03880, 0.01054]	0.5104
two concepts	TwoLayerCNN	early annealing	-0.04199	[-0.06889, -0.01509]	0.0116
two concepts	TwoLayerCNN	default annealing	-0.04156	[-0.06983, -0.01329]	0.0144
two concepts	TwoLayerCNN	late annealing	-0.04523	[-0.06854, -0.02192]	0.00147

C.6 Schedule/task sensitivity endpoints at epoch 200

The comparison below complements the activation-patching results with test accuracy and first-layer filter-template similarity. Values are means over 50 new replicates.

Table 16: Schedule/task sensitivity test accuracy and first-layer filter-template similarity for default annealing versus constant regularization.

Task	Architecture	Schedule	Test accuracy	Template similarity
single shape	TinyCNN	constant $\lambda = 1$	99.98%	0.958
single shape	TinyCNN	default annealing	99.99%	0.682
single shape	TwoLayerCNN	constant $\lambda = 1$	99.98%	1.000
single shape	TwoLayerCNN	default annealing	99.99%	0.942
two concepts	TinyCNN	constant $\lambda = 1$	98.05%	0.944
two concepts	TinyCNN	default annealing	99.45%	0.574
two concepts	TwoLayerCNN	constant $\lambda = 1$	99.11%	0.999
two concepts	TwoLayerCNN	default annealing	99.26%	0.896

C.7 Independent reimplementaion of checkpoint evaluation

The pre-specified reimplementaion covered 32 earlier checkpoint evaluations. All 32 completed after correcting an error in the independent implementation that initially ranked post-second-layer rather than first-layer activations in TwoLayerCNN. The correction did not alter the pre-specified protocol. All recomputed values then fell within the stated tolerances.

Table 17: Discrepancies between the original and independently reimplemented checkpoint evaluations. Full-channel and no-op probability errors were both zero.

Metric	Maximum absolute discrepancy	Tolerance
ordinary accuracy	0	10^{-12}
filter-template similarity	2.22×10^{-16}	10^{-6}
concept AUROC	0	10^{-9}
localization IoU	0	10^{-9}
patching effect for selected channels	2.39×10^{-7}	2×10^{-6}
patching effect for random channels	1.14×10^{-7}	2×10^{-6}
U_{random}	2.19×10^{-7}	2×10^{-6}
patched-run accuracy	0	10^{-12}

D Earlier analyses motivating the pre-specified contrast

These earlier analyses precede the pre-specified channel-count experiment reported in Section C. They are retained to show why the primary analysis used the patching effect for selected channels rather than the selected-minus-random comparison.

D.1 Early 40-epoch comparisons

The initial 40-epoch analysis compared constant template regularization (`template_retention_1`) with template initialization only (`template_init`) at 40 epochs. Four U tests were specified before the outcomes were inspected and were Holm-adjusted as one family. Filter-template similarity and accuracy are also reported descriptively.

Table 18: Early four-comparison analysis. The positive marginal interval in the first row does not survive Holm adjustment.

Task	Architecture	Δ template similarity	Δ acc. (pp)	ΔU [95% CI]	Holm p
single shape	TinyCNN	+0.268	-0.23	+0.0345 [0.0018, 0.0672]	0.1633
single shape	TwoLayerCNN	+0.172	-0.04	-0.0092 [-0.0513, 0.0329]	1.0000
two concepts	TinyCNN	+0.289	-6.13	+0.0252 [-0.0306, 0.0811]	0.9995
two concepts	TwoLayerCNN	+0.198	-0.39	-0.0074 [-0.0557, 0.0408]	1.0000

D.2 Long-horizon epoch-200 condition means

The table below gives the earlier epoch-200 quantities most relevant to the constant-versus-annealed regularization comparison. Means are over ten replicates. The full five-condition tables and checkpoint trajectories remain in the analysis artifact.

Table 19: Earlier epoch-200 means for constant and annealed template regularization.

Task	Architecture	Condition	Accuracy (%)	Template similarity	U_{random}
single shape	TinyCNN	constant $\lambda = 1$	99.961	0.959	0.358
single shape	TinyCNN	annealed	100.000	0.690	0.335
single shape	TwoLayerCNN	constant $\lambda = 1$	99.922	1.000	0.067
single shape	TwoLayerCNN	annealed	99.961	0.942	0.060
two concepts	TinyCNN	constant $\lambda = 1$	98.008	0.947	0.657
two concepts	TinyCNN	annealed	99.219	0.580	0.718
two concepts	TwoLayerCNN	constant $\lambda = 1$	99.141	0.999	0.061
two concepts	TwoLayerCNN	annealed	99.531	0.907	0.003

On `two_concepts`/TinyCNN, patched-run accuracy on the counterfactual target is 93.61% under constant regularization and 90.64% under annealed regularization even though U_{random} is higher under annealing. This discrepancy motivated separate reporting of the selected-channel and control-channel patching effects.

D.3 Post-hoc decomposition of the original selected-minus-random comparison

This post-hoc analysis reused the 80 earlier epoch-200 checkpoints from the annealed- and constant-regularization conditions and evaluated three validation-only channel-selection rules at $k \in \{1, 2, 4, 8\}$. The TinyCNN results are reproduced below. Each entry is annealed minus constant regularization; intervals are marginal paired 95% t intervals over ten replicates. This analysis is post-hoc and is not used as the primary pre-specified test.

Table 20: Post-hoc `two_concepts`/TinyCNN decomposition.

Channel selection	k	ΔF_{sel}	ΔF_{rand}	ΔU	Δ CF acc.
activation difference	1	-0.375	-0.040	-0.335	-0.255
activation difference	2	-0.163	-0.062	-0.101	-0.185
activation difference	4	-0.022	-0.083	+0.061	-0.030
activation difference	8	+0.006	-0.033	+0.040	+0.013
AUROC	1	-0.332	-0.040	-0.292	-0.230
AUROC	2	-0.188	-0.062	-0.125	-0.218
AUROC	4	-0.022	-0.083	+0.061	-0.031
AUROC	8	+0.007	-0.033	+0.040	+0.015
validation patching	1	-0.277	-0.040	-0.237	-0.194
validation patching	2	-0.122	-0.062	-0.060	-0.147
validation patching	4	-0.015	-0.083	+0.067	-0.021
validation patching	8	+0.005	-0.033	+0.038	+0.013

When channels are ranked by activation difference, $\Delta F_{sel}(4) = -0.022$ with 95% interval $[-0.054, 0.010]$, while $\Delta F_{rand}(4) = -0.083$ with interval $[-0.097, -0.068]$. The positive $\Delta U(4) = +0.061$ is therefore partly due to the change in the random-channel baseline. At $k = 8$, the patching effect for selected channels itself becomes slightly positive: $+0.006$ $[0.004, 0.009]$. AUROC and validation-patching channel selection show the same broad pattern.

For `two_concepts`/TwoLayerCNN, the patching effect for selected channels at $k = 4$ is negative under all three channel-selection rules (approximately -0.125, -0.121, and -0.087 for activation difference, AUROC, and validation patching, respectively). Under validation-patching selection the score becomes positive at $k = 8$ ($+0.0215$ with marginal interval $[0.0049, 0.0381]$). These outcomes motivated the later pre-specified contrast between smaller ($k = 1, 2$) and larger ($k = 4, 8$) channel sets rather than a hypothesis about the sign at any single k .

D.4 Validation-energy-matched control

A later post-hoc control chose alternative channel sets of the same size whose validation replacement energy was closest to that of the selected set. All 80 checkpoint evaluations were completed. Matching quality varies substantially with channel count: single-channel sets can have large relative mismatch, while $k = 4$ and especially $k = 8$ are generally tightly matched in the `two_concepts`/TinyCNN setting. Changing the baseline does not alter the annealed-minus-constant effect for the selected channels, which is the quantity used to motivate the pre-specified channel-count experiment. The comparison with the energy-matched baseline is therefore reported as a secondary analysis.

D.5 Transition to the pre-specified channel-count experiment

The earlier analyses showed that constant template regularization preserves high filter-template similarity, that prediction can improve after the regularizer is annealed to zero, and that the annealed-minus-constant patching comparison differed across the tested channel counts. Because the original sign change in U combined changes in both the selected-channel and baseline patching effects, the channel-count protocol was specified around the patching effect for selected channels before the new replicates were generated. Section C reports that pre-specified test and the subsequent secondary sensitivity analysis over 50 new blocks per task/architecture setting (1,200 trained models in total).

E Complete filter-matching table

The revised main text uses the one-to-one matching analysis only as a compact structural robustness check. This appendix reports the complete epoch-200 numerical table; the archived research artifact also retains the matching trajectories, full similarity matrices, and matched-filter gallery from the earlier manuscript version.

All values below are epoch-200 means $\pm$ sample standard deviation across ten replicates; conditions are paired within replicate. The nearest-template and one-to-one scores use signed cosine similarity. Coverage counts distinct nearest-template entries; because the bank has rank 10, it is neither a count of linearly independent directions nor a count of independent semantic concepts. The analysis uses only saved checkpoints from the earlier analyses. These descriptive comparisons are exploratory.

Table 21: Epoch-200 filter-template matching results.

Task	Architecture	Condition	Nearest-template	One-to-one	Coverage
single shape	TinyCNN	random	0.300 ± 0.023	0.267 ± 0.033	9.700 ± 1.703
single shape	TinyCNN	unit-norm random	0.288 ± 0.022	0.256 ± 0.034	9.700 ± 1.703
single shape	TinyCNN	initialization only	0.610 ± 0.021	0.605 ± 0.021	13.300 ± 1.337
single shape	TinyCNN	annealed regularization	0.690 ± 0.017	0.686 ± 0.017	14.200 ± 1.033
single shape	TinyCNN	constant $\lambda = 1$ regularization	0.959 ± 0.003	0.959 ± 0.003	16.000 ± 0.000
single shape	TwoLayerCNN	random	0.261 ± 0.037	0.230 ± 0.039	9.200 ± 1.317
single shape	TwoLayerCNN	unit-norm random	0.228 ± 0.033	0.195 ± 0.039	8.300 ± 1.889
single shape	TwoLayerCNN	initialization only	0.799 ± 0.015	0.799 ± 0.015	15.700 ± 0.483
single shape	TwoLayerCNN	annealed regularization	0.942 ± 0.006	0.942 ± 0.006	16.000 ± 0.000
single shape	TwoLayerCNN	constant $\lambda = 1$ regularization	1.000 ± 0.000	1.000 ± 0.000	16.000 ± 0.000
two concepts	TinyCNN	random	0.232 ± 0.034	0.199 ± 0.035	9.100 ± 1.524
two concepts	TinyCNN	unit-norm random	0.222 ± 0.031	0.191 ± 0.037	9.200 ± 1.398
two concepts	TinyCNN	initialization only	0.507 ± 0.022	0.496 ± 0.022	12.000 ± 1.054
two concepts	TinyCNN	annealed regularization	0.580 ± 0.011	0.574 ± 0.010	12.900 ± 0.994
two concepts	TinyCNN	constant $\lambda = 1$ regularization	0.947 ± 0.004	0.947 ± 0.004	16.000 ± 0.000
two concepts	TwoLayerCNN	random	0.255 ± 0.027	0.231 ± 0.024	9.600 ± 1.075
two concepts	TwoLayerCNN	unit-norm random	0.243 ± 0.028	0.218 ± 0.027	9.000 ± 1.054
two concepts	TwoLayerCNN	initialization only	0.746 ± 0.017	0.746 ± 0.017	15.400 ± 0.699
two concepts	TwoLayerCNN	annealed regularization	0.907 ± 0.012	0.907 ± 0.012	16.000 ± 0.000
two concepts	TwoLayerCNN	constant $\lambda = 1$ regularization	0.999 ± 0.000	0.999 ± 0.000	16.000 ± 0.000